Talk Script (English): Feeding Coding Agents with a memory-mcp Built from Multi-Repository Docs

Oct 13, 2026 · @jin

Overview

This talk is about gathering dev docs scattered across many repositories into a meta-repository, then feeding them to coding agents through an OpenSearch-based memory-mcp, so we stop re-explaining context at the start of every session. It is written for a 50-minute talk plus 10 minutes of Q&A, with the expected time for each section in parentheses.

Item	Content
Audience	OpenSearch Korea User Group. Engineers who have run OpenSearch
Starting point	Context is something you own. History should work like phone numbers: you don't memorize them all, you look them up when you need them. This is the story of building that contact list for a whole team, in four stages
Key message 1	OpenSearch comes in at stage 3, not at the start. A meta-repository directory structure and INDEX.md are enough to begin
Key message 2	Accumulated docs always weather. Without a 60-day lifecycle and compaction, you just burn tokens
Key message 3	Episodic docs and knowledge docs need different storage and different retrieval. Episodic goes to OpenSearch, knowledge goes to a graph DB
Delivery	Conversational and polite. Text in square brackets `[ ]` is a stage direction and is not read aloud

Slide numbers in this script match the English deck, which keeps the same 37 slides in the same order as the Korean deck.

1. Opening: context is something you own (2 min)

(Slide 1: Cover)

Hi everyone. I'm Jin, and I build the data platform for the ads group at Karrot (Daangn). We collect attribution data and produce the data that ad serving runs on. Our schemas are tied to the services, so I usually have six or seven repositories open at once.

(Slide 2: "Own your intelligence")

Today's story starts with one line. It's from Garry Tan, the CEO of Y Combinator, in his Startup School keynote this August: "Own your intelligence." The gist fits in three sentences. The model is rented, and context is owned. The better the models get, the more the differentiator moves to context. And every model release is a free upgrade on top of the context you already own.

Today I want to talk about how a team, not just one person, can own that context.

2. If you had joined three months ago (2 min)

(Slide 3: The question)

Let me ask you something. Say you joined your team three months ago. Someone walks up, points at a decision the team made sometime in the past year, and asks why it was done that way. What do you say?

[Pause]

Probably "I wasn't here back then." If you're feeling generous, "I'm sure there was a reason?" [Wait for the laugh] So you go looking for that reason. And is your team's documentation by any chance... Slack messages? You search, a few threads come up, and the reason is scattered somewhere between them.

If that's hard for people, think about a coding agent that starts every session with an empty memory. For the agent, every session is its first day on the job.

3. History should work like phone numbers (1 min)

(Slide 4: Phone numbers)

How many phone numbers do you actually know by heart? Your wife's, of course. [Laugh] But your team lead's? You don't need to. You know it starts with 010, like every mobile number in Korea. [Laugh] When you need it, you look it up in your contacts.

I think history should work the same way. A few things you use every day stay in your head. The rest you don't memorize; you just need to find it, exactly, when you need it. That goes for people, and it goes for agents. Today I'll talk about building that contact list for a whole team.

4. How do you manage your dev docs? (3 min)

(Slide 5: One question)

I have a question for you. When you work with coding agents, dev docs and spec docs keep piling up. How are you all managing them?

[Ask for a show of hands, one option at a time]

- Anyone running a meta-repository?
- Or do you write a spec doc for each task and keep it inside that repository?
- Or is your philosophy that AI-generated docs are just leftovers, and the single source of truth is always the source code?

[Read the room for a moment]

Those of you who raised your hand for the third one: I get it. Everyone has their own philosophy. Today I'll just tell you about mine.

(Slide 6: What I dealt with every day)

Every time I handed a task to an agent, I had to write out the context: "this project has this background, this term means this, last time we decided that." It was repetitive, and it wore me out. I'll tell you what I did to get rid of that fatigue, and where OpenSearch came into it.

5. Why one repository isn't enough (3 min)

(Slide 7: What the team does)

I'm on the data platform team in our ads group. What we do fits in two sentences. We collect ad attribution and serve it. And we build active-ad data and hand it to the serving team. The second one, building active-ad data, is where most of the business logic lives.

To reason correctly about the side effects of my changes, I need to know which service code this data came from, and how. But our data schemas are still tightly coupled to the service layer. So I can't look at just one repository. I have to look at several at once.

(Slide 8: My everyday situation)

Concretely, it looks like this.

- I have to read code in repositories I've never contributed a single line to.
- I have to know which direction other teams' repositories are moving.
- And I have to explain all of that to the agent from scratch, every session.

The third one was the real problem. If I skipped the explanation, the agent would invent history that never happened and build on it. By the time I'd fixed that, the day was gone.

So I thought: let's build one repository that tracks the state and direction of all the others in one place. That was the start of the meta-repository.

6. What I wanted, and the four stages (3 min)

(Slide 9: What I wanted)

What I wanted from this meta-repository was really just one thing.

I wanted the AI to understand that repository's history on its own.

For example, I wanted the terms we use in planning to map exactly onto the actual code. The planning doc says "active ad," the code calls it something else, and I'm the only one who knows how it got from one to the other. I'm sure most of you have been there. So the requirements come down to two lines.

- Don't make me explain past history every session, and don't make up history that doesn't exist.
- When work on a repository starts, go read its past history on your own.

(Slide 10: Four stages)

One thing up front. I didn't start by building a memory-mcp. It took four stages to get here, and each stage came out of a problem the previous one left behind.

1. I had to re-explain history every session, so I built a meta-repository and a directory structure. Up to this point there was no OpenSearch, just INDEX.md files.
2. Docs also lived outside, and the ones that piled up went stale, so I started importing outside docs and gave docs a lifecycle.

3. The meta-repository got so big that exploring it took forever. Only then did I add an OpenSearch-based memory-mcp.
4. Facts and valid rules had to stay current at all times, so I moved knowledge into a graph DB.

Let's go through them one by one.

7. Stage 1: the meta-repository and its directory structure (6 min)

(Slide 11: meta-repository)

Have you heard the term meta-repository? It's simple. It's one repository that collects the meta information of many repositories.

What counts as "meta information" is the important part. I settled on four things: the repository's specs, its ADRs, its PRDs, and the plan documents the AI writes before it starts working. The agent saves these into the meta-repository, and if the original repository already has a docs directory, the guide tells it to pull information from there.

(Slide 12: Directory structure)

Here's what the meta-repository I actually run looks like.

```
meta-repo/
├── CLAUDE.md
├── repos/                          # managed repositories (checked out
alongside)
└── docs/
    ├── repository/{repo}/
    │   └── INDEX.md
    ├── feature/{yyyy-mm-dd}_{topic}/
    │   └── INDEX.md
    ├── specs/{yyyy-mm-dd}_{spec}/
    │   └── INDEX.md
    └── knowledge/{topic}/
        └── INDEX.md
```

Under repository go that repository's ADRs, background, and term mappings. feature holds features that span several repositories. specs holds specs written by people, specs generated by the AI, and work plans. knowledge holds only facts and rules, by topic.

Every directory has an INDEX.md. For people it's the entry point; for the agent it's the table of contents it reads before reading everything else. Thanks to INDEX.md, we could run the whole thing early on without OpenSearch at all. Later, once the search engine came in, we no longer needed to maintain INDEX.md.

(Slide 13: Each directory has a different premise)

What matters in this structure is the premise behind each directory.

Every subdirectory of feature and specs is named {yyyy-mm-dd}_{topic}. Almost everything inside is episodic: a record of what happened on that day and what was decided. So it's tied to a date.

Everything under repository, on the other hand, assumes it is always current. It's what that repository looks like right now.

knowledge subdirectories are just topics. They contain only facts and rules, so they have no date and no lifecycle. This split is the foundation for the stage 2 lifecycle and the stage 4 knowledge graph.

(Slide 14: worktree and CLAUDE.md)

The managed repositories are checked out inside the meta-repository. Work always happens in a fresh worktree, for three reasons.

- When the agent runs several tasks at once, branches get mixed up; with worktrees, each task gets its own directory.
- The working path is fixed, so you can trace which change came from which task.
- codebase-memory-mcp, which I'll come back to later, detects worktrees and answers where code lives.

Then I wrote a CLAUDE.md. It tells the agent how to use this repository. When starting work, read that repository's INDEX.md first; from stage 3 on, call search_memory first. Save result docs into this directory with this frontmatter. And the change history of these rules. With that history, the agent can tell which docs were written under the old rules.

One more thing. Whenever we generate information from docs, the instruction always includes the words "without inference." That one phrase made a bigger difference than I expected.

(Slide 15: Skill)

Then I built a skill. Give it a Jira or Linear ticket and it does the work, writes the docs, moves the ticket status, sends a Slack notification, designs black-box tests and posts them as a Linear comment, deploys, and even runs the e2e tests, all automatically.

It sounds impressive, but I built it pretty roughly. Sloppily, actually. I just asked the AI to make it. [Laugh] What matters is step three: when the work is done, a doc is left in the meta-repository. Writing docs stops being homework for people and becomes a by-product of the work.

But once docs started piling up, two things became clear. Docs also lived outside the meta-repository. And the docs that piled up went stale over time.

8. Stage 2: import docs, give them a lifecycle (3 min)

(Slide 16: Importing docs)

Docs come in three ways: when we build or change a feature, when we sync Notion docs, and every doc in the repositories we manage. Everything that comes in goes to feature and piles up as episodic. Nothing goes straight into knowledge. Only the facts and rules that survive the lifecycle I'm about to describe make it into knowledge.

(Slide 17: The document lifecycle)

A repository's direction and details weather over time. Docs that were no longer valid just stayed there, and the agent burned tokens wading through far too many of them.

So I gave docs a lifecycle. The target is always episodic docs older than 60 days, meaning feature and specs. The 60 days are counted by the age of the document.

1. For docs past 60 days, create a new document that stores the compacted result.
2. That result is added to feature or knowledge as a new doc, or appended to an existing one, along with the S3 path of the original.
3. The originals past 60 days move to S3, our remote storage.
4. All of these changes are opened as a PR.

The key is that nothing gets deleted. Originals go cold; what survives stays hot. And knowledge never goes through the lifecycle. Once a fact or rule is in, it doesn't go cold no matter how old it gets.

[These words may sound different to OpenSearch people, so clarify once] Compaction here doesn't mean segment merging. It means rewriting documents to make them smaller. And cold doesn't mean a data tier. It means archived in S3. This is where the idea of hot

and cold first appeared, but at this point hot and cold were still files and S3 paths, not OpenSearch.

But by the time we got through stage 2, a new problem showed up. The meta-repository had grown too big.

9. Stage 3: the OpenSearch memory-mcp (9 min)

(Slide 18: Why we moved to stage 3)

When several docs use the same words and similar sentences, the agent reads all of them. It walks the file tree, searches with ag, opens, reads, compares. That repeats for every similar doc, and the time and tokens add up.

That part is exactly what search engines have been doing for decades: information retrieval, IR. So instead of the agent indexing and searching by itself, let OpenSearch handle the IR and have the agent just take the results. That's stage 3.

(Slide 19: Requirements)

What I needed was clear. Pull knowledge out of episodic docs that keep piling up in time order. At this stage I gave up on building one perfect knowledge base. Managing conflicts over a single resource, "is it A or B," "merge A and B into C," was too heavy to take on. I had four conditions.

#	Requirement	Why
1	Put doc files from GitHub or local disk into OpenSearch	The docs already exist as files in the meta-repository
2	Keep docs separated by team and by group	Even inside the ads group, each team has its own context
3	Have hot and cold areas	We may need context from a year or two ago. I wanted old history to stick around too
4	Store cold in S3 and load it straight back into OpenSearch when needed	Too valuable to delete, too expensive to keep online all the time

[To the audience] You might ask why not SQLite or PostgreSQL. I did think about it. But if you know how to run OpenSearch, there's no reason to pick something else. I suspect most of you here would reach the same conclusion.

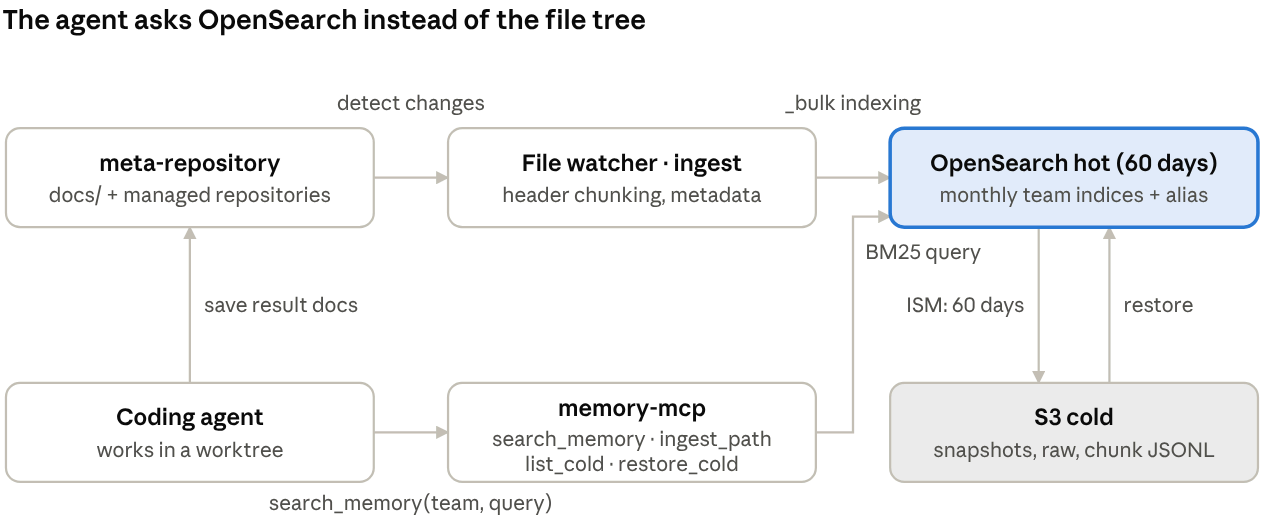
[Not on the slide. Say it briefly] First I looked at what others had already built. The memory tools in the official OpenSearch MCP server, community memory MCPs, local markdown RAG MCPs on SQLite: indexing and querying were all well covered. Still, I skipped the official server and built four tools myself. Honestly, the reasons were these. Because I could. I wanted team and org concepts in it. I didn't know the official server well. And I wanted to learn. Once I built them, the tools turned out smaller than I expected.

(Slide 20: Six features)

1. OpenSearch runs on my local machine first. A shared team cluster is a later problem.
2. Put everything in, for now. The meta-repository's docs and the managed repositories' docs are all indexed.
3. Watch the files. Any change under docs gets re-indexed.
4. Start with BM25 only, no embeddings. It's a cost thing. Embeddings come when we move to remote mode.
5. Only hot goes into OpenSearch. The cutoff is the same 60 days from stage 2.
6. When cold is needed, download it from S3 and load it back into OpenSearch.

Number 2 is how I honestly felt. You usually just throw everything in. My thinking was that however badly I built it, it would still beat searching with ag. [Laugh]

(Slide 21: Architecture)



memory-mcp architecture · top row: how docs get indexed, bottom row: how the agent asks

The top row is how docs get indexed, and the bottom row is how the agent asks. Indices older than 60 days are sent to S3 by ISM, and come back via restore only when needed.

Does BM25 alone work? It does. The one searching isn't a person, it's an agent, so the queries it sends are natural-language sentences, and BM25 scores those tokens. Dev docs in particular are full of identifiers, like function names, service names, error codes and file paths, which are strong lexical signals, so BM25 fits well a lot of the time. When the words don't match, the agent can rephrase and call again two or three times, and that happens naturally in the MCP tool-call loop.

(Slide 22: hot/cold setup)

How did we implement hot/cold? Episodic docs are append-only and pile up in time order, so they behave just like log data. So we borrowed the log operations pattern as is.

Layer	Setup
Index	`mem-{team}-{yyyy.MM}` monthly indices. A read alias points at all hot indices. Requirement 2 (per-team separation) is handled by index names and aliases
Document fields	team, repo, doc_type (spec/ADR/PRD/plan), event_time, source_path, git_sha, header_path, body text
Chunking	By markdown header hierarchy. Never split inside a code block
ISM policy	hot for 60 days → snapshot to an S3 repository → delete the index. When needed, restore the snapshot to reload a whole index
S3 layer	Raw originals, parsed chunk JSONL, snapshots. When we change mappings or add embeddings later, re-load from the JSONL with _bulk

The analyzer is nori. We set nori, the Korean morphological analyzer, as the index's default analyzer, because our docs mix Korean text with planning terms. And knowledge docs are not under ISM. They don't go through the lifecycle.

(Slide 23: MCP tools)

Four MCP tools were enough.

Tool	Role
`search_memory(team, query, time_range)`	BM25 search on the hot alias. Results are chunks that include source_path and header_path
`ingest_path(path, team)`	Manual ingest for anything the watcher missed
`list_cold(team)`	Lists the cold indices sitting in S3
`restore_cold(index)`	Snapshot restore. When the agent gets a query in the cold range, it first asks "there's a cold index X, should I restore it?"

10. Demo and what we saw (6 min)

(Slide 24: Demo)

[Switch to the demo screen]

Ta-da. Right now a memory-mcp is running that has the docs from docs loaded into OpenSearch and searches them, and the agent refers to these docs every time it works. Let's reuse the "active ad" example from earlier.

[Demo steps]

1. Save one ADR with the "active ad" term mapping under the meta-repository's docs. Show the log of the watcher picking it up and indexing it.
2. Open a fresh agent session and give it a task related to "active ad." No background explanation.
3. Show the agent calling `search_memory` and working from the `source_path` of the chunks it got back.
4. Put it side by side with the same task run without memory-mcp. There are no measured numbers, so let the difference on screen speak.

(Slide 25: Measurement)

To be honest, there are no numbers. I didn't measure it. But you can tell once you use it. (Probably..?) [Pause] The agent spent less time digging through the file tree and opening the same docs over and over, and the time and tokens went down noticeably.

The tools themselves were really easy to build. But while episodic docs could be handled this way, knowledge was a different story.

11. Stage 4: the knowledge graph DB (3 min)

(Slide 26: Why we moved to stage 4)

Almost everything going into memory-mcp right now is episodic. These are records of "what happened in that session on that day," so they carry a timestamp and lose value as the days go by. That's why sending them cold after 60 days doesn't hurt. Knowledge is the opposite: timeless facts and rules, like "this team does it this way." Once it's right, it stays right for a long time, and when there's a conflict, you have to work out which side is correct.

	Episodic docs	Knowledge docs
What it is	Work records that pile up in time order. ADRs, plans, meeting decisions	Facts that were filtered and survived. Term definitions, system invariants
Storage and IR	An inverted-index DB. Today, OpenSearch BM25	A graph DB (neo4j). Remote storage, not files
Lifecycle	Must have one. Hot for 60 days, then cold	Permanent. But needs an update history
Conflicting facts	Doesn't matter. Different records on the same topic coexisting is normal	Always a concern. Pick A or B, or create C
Flow	Facts that survive compaction move on to knowledge	Comes in only from episodic

The key is how each one treats conflicts. Episodic doesn't need to care about conflicts, so its design is light, and that's why we could build it first. Knowledge always has to care, so it had to be run separately. That's also why knowledge never goes through the lifecycle.

(Slide 27: Knowledge graph DB)

So in stage 4 we moved knowledge into a graph DB, neo4j. There's one goal: keep facts and valid rules current at all times. The way in is the facts and rules that survive stage 2 compaction.

One thing surprised me once we did it. This kind of information turned out to work better in remote storage than in files. [One or two sentences on why it was better, to fill in before the talk]

(Slide 28: Demo repository)

[Open the repository and show the structure covered so far. What to show: to fill in before the talk]

Everything I've described lives in this repository: github.com/drakejin/workspace.

12. How others do it: context engineering (4 min)

(Slide 29: Same problem, Elastic's context engine)

That's what I've built so far. But it turns out I'm not the only one working on this. On October 8 this year, Elastic's official blog, Search Labs, published a post called "Context engineering for agents."

Here's the framing. Context engineering so far has focused on what to put into a single agent's context window. The harder problem is building the context before the conversation starts, and letting many agents share it. In one line: "A context engine understands your data ahead of the question."

The three problems the post names are the same ones we ran into. Per-query discovery cost, we addressed with INDEX.md in stage 1 and OpenSearch in stage 3. Context drift, meaning hand-written docs going stale, we addressed with the stage 2 lifecycle. Siloed knowledge, where what one agent learned can't help another, we haven't solved yet. Our OpenSearch still lives on my local machine.

Elastic also published numbers. Same question, same model: tool calls went from 33 to 9, input tokens dropped 71%, and response time went from 213 seconds to 64. As I said earlier, we didn't measure ours. Probably similar, right? [Laugh] The data domains are different too, so just look at the direction.

(Slide 30: Mapping onto the loop)

Elastic places this context layer between the raw data and the agents, and describes it as a loop of four stages that keeps going around: Gather, Build, Retrieve, and Improve. In the Improve stage, the agents' usage records have to flow back in as a new source. That's what makes it a loop. And they see all four stages as search problems: "If you're designing the context layer for your agents, build it on search." A welcome conclusion for this room. [Laugh]

If I map our four stages onto this loop, Gather and Build are covered. Every source comes into feature, and compaction moves facts and rules into knowledge. But Slack conversations and ticket comments aren't sources yet, and we only have two kinds of knowledge unit, facts and rules. Retrieve is partially covered. We have BM25 with nori, but no semantic search, and since all four tools point at OpenSearch, we can't ask the knowledge graph in the same query. And Improve is empty. People change the rules, and people review the compaction PRs. What the agent failed to find, or whether it never called search_memory at all, never flows back in.

(Slide 31: The missing link)

In short, what we have today is a one-way pipeline, not a loop. Usage has to flow back in for it to become a loop. I see four ways to close the gap.

1. **Usage records as a source.** Log search_memory queries, results and tool calls, and look at them during compaction. Only when what we failed to find is recorded can the next compaction change what it produces.
2. **Measure search quality.** Score BM25 results against a query set and judgment lists, and compare with experiments every time something changes.
3. **A cluster the team shares.** Move the local OpenSearch to a shared team cluster, so what one person's agent learns, the whole team can use.
4. **Ask once.** Query episodic and knowledge in a single request, with semantic search on top of BM25.

The good news is we don't have to build any of this from scratch. OpenSearch already has UBI for recording user behavior, and the Search Relevance Workbench for query sets, judgment lists and experiments. They're even exposed as tools in the official OpenSearch MCP server. The Improve stage the Elastic post talks about, we can fill with OpenSearch.

[Sort out the priority of these four before the talk]

13. What works well together (2 min)

(Slide 32: Wrapping up)

[Section divider. Close in this order: what worked well together, what I want to say, the open question]

(Slide 33: Three things that worked well together)

memory-mcp alone doesn't solve everything. Here are three things that worked well alongside it.

First, codebase-memory-mcp. It answers where code lives, not docs. It keeps the code structure as a graph, so it can answer "which function produces this field?" without walking the file tree. It detects worktrees and answers well there too, though honestly I don't yet know how it detects code changes in a worktree under the hood. [Say this honestly] Split it so docs go to OpenSearch and code goes to codebase-memory-mcp, and the tokens the agent spends scanning the file tree drop a lot.

Second, code history in comments. It can feel like stuffing junk into comments. But if a class member variable carries some strange history, spelling out why in a comment makes the context the agent reads far clearer. Docs live apart from the code, but comments sit right next to it, so the agent reads them the moment it reads that code.

Third, separating episodic from knowledge. This is the split we just saw. If you separate them from the start, both the hot/cold cutoff and conflict handling get much simpler.

14. Takeaways and the open question (3 min)

(Slide 34: Five things)

One. Start with a meta-repository. A directory structure and INDEX.md are enough to begin. Agents can't find docs that are scattered across repositories. And when they can't find something, they make it up.

Two. Give your docs a lifecycle. That's what keeps docs from becoming leftovers. That's my answer to those of you who raised your hand for "docs are leftovers" at the start. Keep processing them, compressing them, keeping them current. Small structures like frontmatter, header_path and INDEX.md decide your search quality. And once your docs get big, leave IR to OpenSearch, not the agent. Everyone in this room already runs that tool.

Three. Run codebase-memory-mcp locally on the codebases you manage. It saves a lot of the tokens and time spent exploring and scanning files every session.

Four. Memory design decides how well an agent performs. Same model, same prompt, and the result changes completely depending on what context comes in front of it. This isn't just my own experience; it's something the industry keeps saying. [Fill in two quotes in the format "Quote" — quoted from whom. Source: ...]

Five. Run knowledge in a graph DB. Facts and rules fit remote storage better than files.

(Slide 35: The open question)

Once you run knowledge, you hit one problem right away. When a fact coming from episodic collides with an existing fact, who decides whether it's A, B or C? My current answer is simple. On a conflict, the latest wins, and if the latest looks false, a person reviews it. But who decides that something "looks false," and how, is still an open hole.

(Slide 36: Thank you)

So I'll end with that question left open. How would you handle it? I didn't come here to hand out answers; I came to talk it through with you. Thank you. I'm happy to take questions and hear your stories.

(Slide 37: Expected Q&A and references)

[Keep this up during Q&A. The six cards on top are the expected questions from section 15 most likely to land hard, with short answers. Say only as much as the slide shows, then hand it back to the audience. Admit plainly that compaction verification and doc conflicts are still unsolved. The three lines at the bottom link the video, the blog post and the official MCP server I cited]

15. Expected questions and answers

Slide 37 shows six of these with short answers: why not the official MCP server, isn't ripgrep enough, why neo4j for knowledge, verifying compaction, doc conflicts, and what if the agent never calls search_memory.

Expected question	Gist of the answer
Why didn't you use the memory tools in the official OpenSearch MCP server?	There are two options. Memory Tools need Amazon OpenSearch Service 2.19+ or Serverless, so they don't run locally. Agentic Memory works self-hosted on 3.3+, but it's for storing plain-text facts, with no file ingest and no chunking. We skipped the official server and built four tools ourselves. Because we could. We wanted team and org concepts, we didn't know it well, and we wanted to learn. The official server also has a SearchMemoryTool, so keep the two names apart when you answer

Expected question	Gist of the answer
Is search quality OK with BM25 alone, no embeddings?	Dev docs are full of identifiers, which are strong lexical signals, so BM25 fits well. If misses pile up, the plan is to add a `knn_vector` field and move to hybrid queries. Full re-indexing is easy by re-loading from the parsed JSONL in S3
How do you handle Korean docs?	nori is the index's default analyzer. Korean text and planning terms are indexed by morpheme
Why 60 days for the hot/cold boundary?	It's the same cutoff we used when we deleted old docs by hand. It can differ by team, so it's a single number in the ISM policy
Couldn't you search cold directly with searchable snapshots?	Yes, and that's the next option. Cold queries are rare for now, so we started with restore. The key part is that the agent asks the user before restoring
Does the team/group separation give you security too?	Today it's only logical separation by index name and alias. If we need access control, we'd extend it with index-pattern role mapping in the Security plugin
How do you handle doc collisions?	Not at the episodic stage. Different records on the same topic coexisting is normal, and a time filter favors the latest. Only when moving to knowledge does a person decide, during compaction. Automating that is unsolved
What if the agent never calls search_memory?	It's written into CLAUDE.md as a rule for starting work. If it still gets skipped, we're experimenting with a session-start hook that runs one automatic search by repository name and adds the results to the context
How much did you save in tokens?	We didn't measure it. Say honestly that all we have is the feel of it: noticeably less file exploration and fewer repeated reads of the same docs

Expected question	Gist of the answer
Is the code public?	Yes. The repository is github.com/drakejin/workspace (the address is on the demo slide and the last slide)
Why isn't knowledge in OpenSearch? Which graph DB?	neo4j. It felt better to use OpenSearch only as an inverted-index store and not give it any role beyond that. [Fill in before the talk: why remote storage worked better than files]
How does the Notion sync work?	[Fill in the sync method and schedule before the talk]
How is this different from Elastic's context engine?	Elastic builds knowledge units ahead of time from data (indices, contracts); we build them from dev docs. The biggest difference is the Improve stage. Elastic feeds agent records back in as a source; we haven't done that yet
Is the 60 days the document's age or the index's age?	The document's age. [ISM's age condition is based on index creation time, so expect a follow-up on how that lines up with monthly indices]
At that scale, isn't ripgrep enough? What's the basis for BM25 quality?	We started with ripgrep, but judged that searching and exploring docs over and over wasted more tokens. When several docs share the same words, the hunch was that OpenSearch would do better at finding the one with the right nuance
Who verifies information lost or made up in compaction summaries? How much PR review does it take?	We haven't verified it. Say honestly that this is the blind spot of this demo

16. Sources

- Garry Tan: Own Your Intelligence — Y Combinator, Startup School 2026 (video) — The original video for "Own your intelligence" in section 1 (opening). Linked on slide 37 (expected Q&A and references).

- Garry Tan: Personal AGI Is How You Stay Under Your Own Power — Startup School 2026, 2026-08-06 — A write-up of the video above. Source of the section 1 quotes: "The model is rented," "The better the models get, the more the differentiator moves to context," and "a free upgrade."
- Context engineering for agents: 33 tool calls down to 9 — Elastic Search Labs, Anish Mathur, 2026-10-08 — Section 12 (how others do it). Source of the context engine definition, the three problems (per-query discovery cost, context drift, siloed knowledge), the Gather, Build, Retrieve, Improve loop, and the comparison numbers (tool calls 33 → 9, input tokens down 71%, 213 s → 64 s).
- opensearch-project/opensearch-mcp-server-py — GitHub — Section 12, the missing link. Source for the Search Relevance Workbench and UBI tools, and the requirements of the official server's Memory Tools.

The outside quotes come from three places: Garry Tan, the Elastic post, and the official OpenSearch MCP server. Everything else is the speaker's own story. The two quotes for the fourth takeaway in section 14 still need their sources checked and added here before the talk.